

Trust-Aware AI-Assisted Service Placement

in Edge Computing Systems

Pediredla Surya Venkata Mourya

Roll No: 122CS0563

Supervisor: Prof. Dr. Bibhudatta Sahoo

Department of Computer Science and Engineering
National Institute of Technology Rourkela

Research Project – II | March 2026



Outline

- 1 Motivation & Problem
- 2 System Design
- 3 Trust Model
- 4 Machine Learning
- 5 Placement Controller
- 6 Results
- 7 Future Work
- 8 Conclusion

Motivation

Edge Computing

- Tasks processed close to the data source
- Reduces latency, supports real-time systems
- Applications: IoT, Smart Cities, Industrial Automation

Challenge

- Edge nodes are **heterogeneous** – vary in CPU, memory, latency, reliability
- A wrong placement decision $\Rightarrow$ task failure, wasted resources, higher latency

Core Problem

How do we **automatically** identify the most reliable edge node *before* deploying a service on it?

RP2 Answer

Combine **Trust Metrics**
+ **Machine Learning**
into one intelligent placement decision.

Problem Statement

Why existing methods fall short:

Method	Why it fails
Random	Blind to reliability – assigns tasks to any available node
Resource-only	Picks the most powerful node, ignores whether it actually succeeds
Trust-only	Uses past history but cannot predict future behaviour

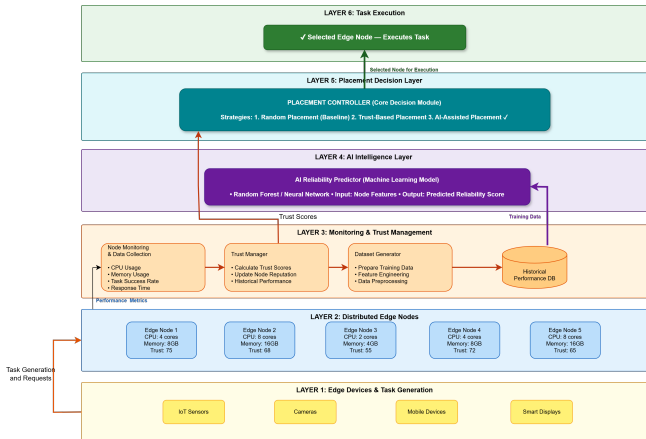
What we need

A placement strategy that:

- ① **Tracks** node behaviour over time
- ② **Learns** reliability patterns from data
- ③ **Predicts** which node will succeed
- ④ **Ranks** candidates using trust + prediction + resources

System Architecture

AI-Assisted Reliability-Aware Service Placement Architecture



Six Layers

- 1 Edge Devices & Task Generation
- 2 Distributed Edge Nodes
- 3 Monitoring & Trust Management
- 4 AI Intelligence Layer (ML)
- 5 Placement Decision Layer
- 6 Task Execution

Trust Management Model

Trust score evaluates the reliability of each edge node: $T_i = \alpha S_i + \beta A_i - \gamma F_i + \delta R_i$

Recent tasks receive higher weight via a recency factor – recent behaviour influences the score more strongly than older events.

Symbol	Meaning
S_i	Successes (recency-weighted)
A_i	Recent activity factor
F_i	Failed tasks
R_i	Success / total ratio
$\alpha = 5$	Gradual trust gain per success
$\beta = 3$	Recency influence (moderate)
$\gamma = 10$	Strong penalty for failures
$\delta = 2$	Small corrective ratio weight

Behaviour

- Reliable nodes **gain** trust over time
- Failures **reduce** trust fast ($\gamma = 10$)
- Inactive nodes **decay** gradually
- Bounded in $[0, 100]$

Threshold Rule

Nodes with $T_i < 60$ are **filtered out**.
Value of 60 chosen empirically
to represent moderate trust level.

ML Implementation – Random Forest

Why Random Forest?

- Handles correlated features (trust & success rate)
- Gives **calibrated probability** via `predict_proba`
- Resistant to overfitting on moderate datasets

Training Pipeline

- 1 Load synthetic CSV dataset
- 2 Min-Max normalise features
- 3 Split: 80% train / 20% test
- 4 Train `RandomForestClassifier` (`n_estimators=100`)
- 5 Deploy: use `predict_proba` at runtime

Input: $X = [CPU, Mem, Lat, T_i, SR]$

Output: $P_i \in [0, 1]$ – reliability probability

Feature Importance

Feature	Score
Trust score T_i	0.38
Success rate SR	0.27
CPU usage	0.16
Latency	0.12
Memory	0.07

Trust score is the **strongest predictor**
– confirms trust and ML are complementary.

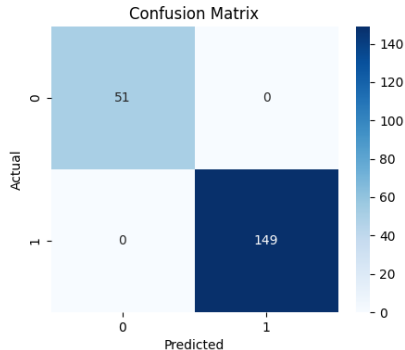
Dataset Generation & Model Evaluation

Synthetic Dataset Attributes

Feature	Description
cpu	CPU utilisation (%)
memory	Free memory (GB)
latency	Round-trip time (ms)
trust	Trust score [0, 100]
success_rate	Historical success ratio
label	1 = Reliable, 0 = Not

Label rule:

$T_i > 60$ **AND** $SR > 0.70 \Rightarrow \text{label} = 1$
Otherwise $\Rightarrow \text{label} = 0$ (Unreliable)



High Classification Accuracy

Model achieves high accuracy on the synthetic test dataset – features are well-separated by design.

Placement Decision

Final node selected using a composite score: $\text{Score}_i = w_1 T_i + w_2 P_i + w_3 L_i$

Component	Weight	Reason
Trust score T_i	0.4	Primary reliability signal
ML prediction P_i	0.4	Forward-looking reliability
Resource score L_i	0.2	Capacity (secondary)

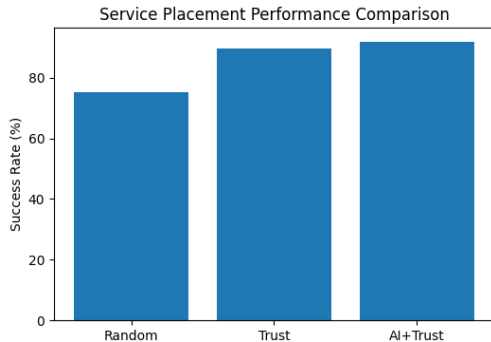
Weights chosen empirically: reliability (trust + ML) dominates at 80%; resources secondary at 20%. A powerful but unreliable node is not useful.

Decision Steps

- 1 Filter: remove nodes with $T_i < 60$
- 2 Score: compute Score_i for remaining
- 3 Rank: sort candidates descending
- 4 Select: $\arg \max(\text{Score}_i)$

Future work will optimise weights automatically using reinforcement learning or Bayesian methods.

Experimental Results



Strategy	Success Rate
Random placement	≈75%
Trust-based	≈88%
AI-assisted	≈92%

- Trust-only is retrospective – misses future failures
- ML adds **predictive reliability** on top of trust
- AI-assisted consistently selects the best node

System Performance

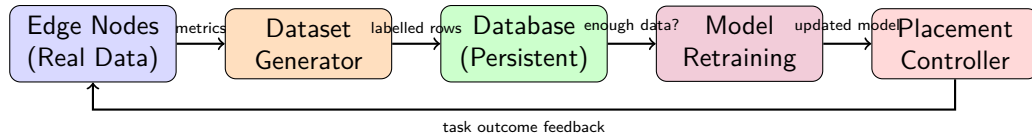
Operation	Latency	Complexity
Trust score update	$< 1 \text{ ms}$	$O(1)$
ML inference	$< 2 \text{ ms}$	$O(1)$
Placement decision	$< 10 \text{ ms}$	$O(n \log n)$
Dataset update	$< 3 \text{ ms}$	$O(n)$

All operations within real-time limits for small-to-medium edge networks.

Memory footprint below **10 MB** for 10-node networks – suitable for resource-constrained edge devices.

Future Work – Continuous Learning Pipeline

Key Extension: ML model learns while the system is running



How it works

- Dataset Generator collects real metrics continuously
- Records stored in a persistent DB
- Enough data $\Rightarrow$ **trigger retraining automatically**

Why it matters

- Current model trained on **synthetic** data only
- Real data captures patterns simulation cannot
- System becomes **self-improving over time**

Other Future Extensions

Simulation & Deployment

- Large-scale evaluation using **iFogSim**
- Deploy on **Raspberry Pi / NVIDIA Jetson**
- Measure real-world latency & power

Automatic Weight Optimisation

- **Reinforcement Learning** for adaptive placement weights
- **Bayesian optimisation** for trust formula constants
- Genetic algorithms for parameter tuning

Advanced ML

- **LSTM** for temporal trust prediction
- Online learning – per-transaction model updates
- Evaluate on larger real-world datasets

Scalability

- Test with 50+ nodes
- REST API for edge platform integration
- Security: digital signatures & encryption

Conclusion

- Designed a **trust-aware AI-assisted** service placement framework for edge computing
- Implemented a **trust scoring model** to evaluate edge node reliability
- Generated a **synthetic dataset** representing realistic edge node behaviour
- Trained a **Random Forest model** to predict node reliability from system metrics
- Built a **placement controller** combining trust, ML prediction, and resource metrics
- Simulation results show **improved task success rate** compared to baseline strategies
- Future: **continuous learning pipeline** – model improves on real-world data automatically

Thank You – Questions Welcome